MSDM 5003 Lecture 10

6 November 2025

Boltzmann Machines and RBM

Outline:

1. Markov random fields (MRF)
2. Boltzmann machines
3. Restricted Boltzmann machines (RBM)
4. Application of RBM to recommender system

1. Markov Random Fields

Consider an undirected graphical model with a set of random variables (0 or 1) defined on the nodes. It satisfies the following Markov properties:

The key idea of Markov properties is conditional independence:

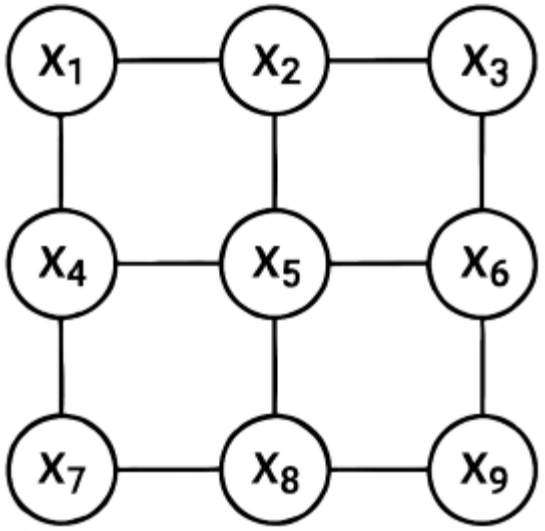
- A variable is conditionally independent of all others given its neighbors.
- This means that to understand a variable, you only need to look at its immediate connections—not the entire system.

Two Markov properties:

Local Markov Property: Each variable depends only on its neighbors.

Pairwise Markov Property: Non-connected nodes are conditionally independent given the rest.

Applications: e.g. image processing



2. Boltzmann Machine[1]

In the Boltzmann machine, the probability of the random variables are modeled as

$$P(x_1, \cdots, x_N) = \frac{1}{Z} e^{-\frac{E(x_1, \cdots, x_N)}{T}},$$

where E is the energy given by

$$E = -\left(\sum_i w_{ij} x_i x_j + \sum_i \theta_i x_i \right).$$

x_i are random variables taking the values 1 or 0.

w_{ij} is the connection strength between node i and node j.

θ_i is the bias of node i.

T is the temperature.

$Z = \sum_{\mathbf{x}} e^{-\frac{E(\mathbf{x})}{T}}$ is called the partition function.

[1]Hinton, Geoffrey E. (2007-05-24). "Boltzmann machine". Scholarpedia. 2 (5): 1668.

Marginal Probability

Consider the marginal probability of node i.

$$P(x_i = 1) = \frac{1}{Z} e^{-\frac{E(x_1,\cdots,1,\cdots,x_N)}{T}}, \qquad P(x_1 = 0) = \frac{1}{Z} e^{-\frac{E(x_1,\cdots,0,\cdots,x_N)}{T}}.$$

Dividing,

$$\frac{P(x_i = 1)}{P(x_i = 0)} = \exp\left\{-\frac{1}{T}\left[E(x_1, \cdots, 1, \cdots, x_N) - E(x_1, \cdots, 0, \cdots, x_N)\right]\right\}.$$

Focusing on the terms containing x_i, $\left[E(x_1, \cdots, 1, \cdots, x_N) - E(x_1, \cdots, 0, \cdots, x_N)\right]$

$$= -\left(\sum_{j \neq i} w_{ij} x_j + \theta_i\right) x_i \Big|_{x_i=1} + \left(\sum_{j \neq i} w_{ij} x_j + \theta_i\right) x_i \Big|_{x_i=0} = -\left(\sum_{j \neq i} w_{ij} x_j + \theta_i\right) \equiv \Delta E_i.$$

Since $P(x_i = 1) + P(x_1 = 0) = 1$, we have

$$P(x_i = 1) = \frac{1}{1 + e^{-\frac{\Delta E_i}{T}}}.$$

Learning in Boltzmann Machines

The Boltzmann machine is shown a training set of binary data vectors.

Learning in the Boltzmann machine refers to the process of finding the weights and biases so that the probabilities fit those of the training set.

The components of the training data may only be restricted to the visible nodes of the machine. Those nodes that do not receive inputs from the training data are the hidden nodes.

The learning rule is obtained by maximizing the product $\prod_{\mathbf{x}\in\text{data}} P(\mathbf{x})$, or maximizing the log likelihood.

$$\frac{dw_{ij}}{dt} \propto \sum_{\mathbf{x}\in\text{data}} \frac{\partial \ln P(\mathbf{x})}{\partial w_{ij}},$$

$$\frac{d\theta_i}{dt} \propto \sum_{\mathbf{x}\in\text{data}} \frac{\partial \ln P(\mathbf{x})}{\partial \theta_i}.$$

Learning Rule

$$\frac{dw_{ij}}{dt} \propto \sum_{\mathbf{x}\in\text{data}} \frac{\partial \ln P(\mathbf{x})}{\partial w_{ij}}$$

$$= \frac{\partial}{\partial w_{ij}} \sum_{\mathbf{x}\in\text{data}} \left[\frac{1}{T} \sum_{k<l} w_{kl} x_k x_l + \frac{1}{T} \sum_k \theta_k x_k - \ln \left(\sum_{\mathbf{y}} e^{\frac{1}{T}\sum_{k<l} w_{kl} y_k y_l + \frac{1}{T}\sum_k \theta_k y_k} \right) \right]$$

$$= \sum_{\mathbf{x}\in\text{data}} \left(\frac{1}{T} x_i x_j + 0 - \frac{1}{Z} \frac{\partial}{\partial w_{ij}} \sum_{\mathbf{y}} e^{\frac{1}{T}\sum_{k<l} w_{kl} y_k y_l + \frac{1}{T}\sum_k \theta_k y_k} \right)$$

$$= \frac{1}{T} \sum_{\mathbf{x}\in\text{data}} \left(x_i x_j - \frac{1}{Z} \sum_{\mathbf{y}} e^{\frac{1}{T}\sum_{k<l} w_{kl} y_k y_l + \frac{1}{T}\sum_k \theta_k y_k} \, y_i y_j \right) .$$

The last term is the expected value of $y_i y_j$ when the Boltzmann machine samples state vectors from its equilibrium distribution at temperature T.

Learning in Visible and Hidden Nodes

In summary, the learning rule is given by

$$\frac{dw_{ij}}{dt} = \frac{\epsilon}{T}\left(\langle x_i x_j \rangle_{\text{data}} - \langle x_i x_j \rangle_{\text{model}}\right).$$

Similarly,

$$\frac{d\theta_i}{dt} = \frac{\epsilon}{T}\left(\langle x_i \rangle_{\text{data}} - \langle x_i \rangle_{\text{model}}\right).$$

The learning rule is the same for both visible and hidden nodes.

Hidden nodes enable the machine to capture higher-order features in the data.

$\langle x_i x_j \rangle_{\text{model}}$ involving hidden nodes are updated when the visible nodes are clamped to the training data.

3. Restricted Boltzmann Machine[2]

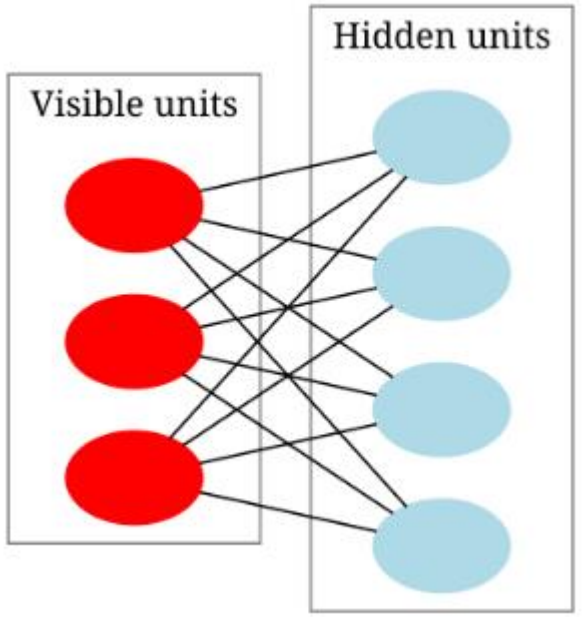
If the visible nodes of the Boltzmann machine are only connected with the hidden nodes, and the hidden nodes only with the visible nodes, then the Boltzmann machine becomes a restricted Boltzmann machine (RBM).

In RBM, the visible nodes are not connected with each other. So are the hidden nodes.

Hence, the structure of RBM is a bipartite graph.

In feature extraction, the hidden nodes can encode the features.

When RBM's are stacked layer by layer, they form deep belief networks, which can extract more sophisticated features. Useful for pre-training deep neural networks.



[2] Restricted Boltzmann machine. (28 June 2025). In Wikipedia. https://en.wikipedia.org/wiki/Restricted_Boltzmann_machine

Formulation

The formulation of RBM is a straightforward extension of that of Boltzmann machines.

$$E(v,h) = -\sum_i a_i v_i - \sum_j b_j h_j - \sum_{i,j} v_i w_{ij} h_j = -\mathbf{a}^{\mathrm{T}}\mathbf{v} - \mathbf{b}^{\mathrm{T}}\mathbf{h} - \mathbf{v}^{\mathrm{T}}\mathbf{W}\mathbf{h}.$$

$$P(v,h) = \frac{1}{Z} e^{-\frac{E(v,h)}{T}}.$$

$$P(v) = \frac{1}{Z} \sum_{\{h\}} e^{-\frac{E(v,h)}{T}}.$$

$$P(v|h) = \prod_{i=1}^{N} P(v_i|h), \qquad P(h|v) = \prod_{j=1}^{M} P\left(h_j|v\right).$$

$$P(v_i = 1|h) = \frac{1}{1 + e^{-\frac{1}{T}\left(a_i + \sum_j w_{ij} h_j\right)}}, \qquad P\left(h_j = 1|v\right) = \frac{1}{1 + e^{-\frac{1}{T}\left(b_j + \sum_i w_{ji} v_i\right)}}.$$

Learning Rule

Same as Boltzmann machines, the learning rule is given by

$$\frac{dw_{ij}}{dt} = \frac{\epsilon}{T}\left(\langle x_i x_j \rangle_{\text{data}} - \langle x_i x_j \rangle_{\text{model}}\right).$$

The first term (also called the positive gradient) is easy to achieve.

The second term (also called the negative gradient), at least theoretically, requires the RBM to iterate until it reaches equilibrium and is thus a computationally expensive operation.

Hinton proposed replacing this operation with Gibbs sampling, in which the network iterates for many steps using MCMC methods.

This requires a "burn-in" period before the network states become representative of the equilibrium distribution.

If there are too few steps, the network states remain strongly correlated with the initial configuration.

Gibbs Sampling[3]

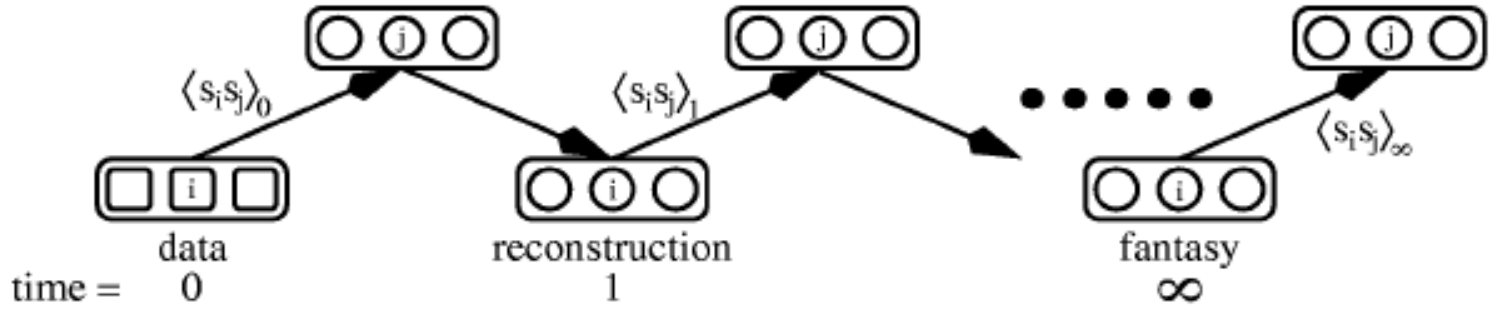


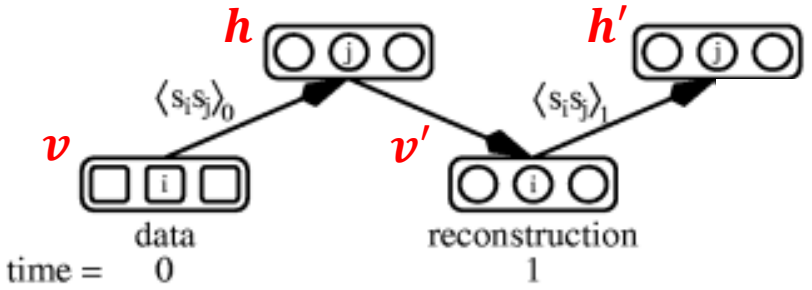
Figure 1: A visualization of alternating Gibbs sampling. At time 0, the visible variables represent a data vector, and the hidden variables of all the experts are updated in parallel with samples from their posterior distribution given the visible variables. At time 1, the visible variables are all updated to produce a reconstruction of the original data vector from the hidden variables, and then the hidden variables are again updated in parallel. If this process is repeated sufficiently often, it is possible to get arbitrarily close to the equilibrium distribution. The correlations $\langle s_i s_j \rangle$ shown on the connections between visible and hidden variables are the statistics used for learning in RBMs, which are described in section 7.

[3] G. E. Hinton, Training product of experts by minimizing contrastive divergence, *Neural Computation* **14**, 1771-1800 (2002).

Contrastive Divergence

Hinton proposed that the first reconstructed state is one step closer to the equilibrium distribution than the original state, so the contrastive divergence can never be negative:

1. Take a training sample v, compute the probabilities of the hidden nodes and sample a hidden activation vector h from this probability distribution.
2. Compute the outer product of v and h and call this the positive gradient.
3. From h, sample a reconstruction v' of the visible nodes, then resample the hidden activations h' from this. (Gibbs sampling step)
4. Compute the outer product of v' and h' and call this the negative gradient.
5. Compute: $\Delta W = \frac{\epsilon}{T}(vh^{\mathrm{T}} - v'h'^{\mathrm{T}})$ and $\Delta a = \frac{\epsilon}{T}(v - v'), \Delta b = \frac{\epsilon}{T}(h - h')$.



4. RBM Recommender System[4]

Collaborative filtering is a recommender system technique that predicts user preferences by analyzing the behavior of similar users or items, e.g. Netflix.

Consider a recommender system with N users and M movies. Each user can give K ratings to each movie. (For example, $K = 5$ in five-star ratings.)

However, each user only gives ratings to a small number of movies.

Building an RBM recommender system:

M visible nodes for the movies.

F hidden nodes representing the features.

The K visible nodes of each movie are softmax units so that they sum up to 1.

$KM \times F$ matrix of weights represent the feature vectors of the movies.

[3] R. Salakhudinov, A. Mnih, G. Hinton, Restricted Boltzmann Machines for Collaborative Filtering, *Proceedings of the 24th International Conference on Machine learning - ICML '07,* 791 (2007).

Missing Ratings

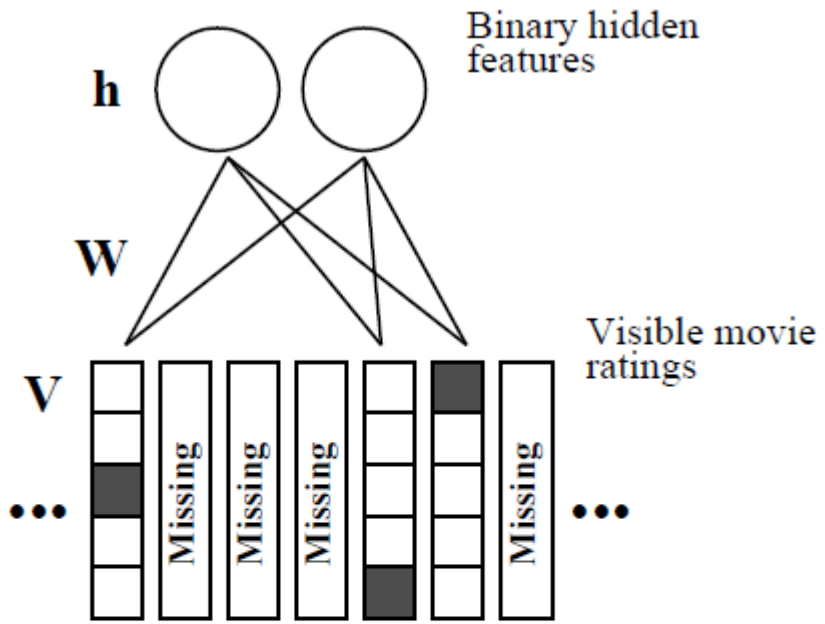


Figure 1. A restricted Boltzmann machine with binary hidden units and softmax visible units. For each user, the RBM only includes softmax units for the movies that user has rated. In addition to the symmetric weights between each hidden unit and each of the $K = 5$ values of a softmax unit, there are 5 biases for each softmax unit and one for each hidden unit. When modeling user ratings with an RBM that has Gaussian hidden units, the top layer is composed of linear units with Gaussian noise.

Formulation (1)

$$p\left(v_i^k \middle| \mathbf{h}\right) = \frac{\exp\left(b_i^k + \sum_{j=1}^{F} W_{ij}^k h_j\right)}{\sum_{l=1}^{K} \exp\left(b_i^l + \sum_{j=1}^{F} W_{ij}^l h_j\right)}.$$

In the RBM built for the user who has rated only m movies,

$$p\left(h_j = 1 \middle| \mathbf{V}\right) = \frac{1}{1 + \exp\left(-b_j - \sum_{i=1}^{m} \sum_{k=1}^{K} v_i^k W_{ij}^k\right)}.$$

$\mathbf{V}$ is the $K \times M$ matrix with $v_i^k = 1$ if the user rated movie i as k and 0 otherwise.

b_i^k is the bias of rating k of movie i. It is initialized to the logs of its base rates of all users.

b_j is the bias of feature j.

.

Formulation (2)

Marginal distribution:

$$p(\mathbf{V}) = \sum_{\mathbf{h}} \frac{\exp(-E(\mathbf{V}, \mathbf{h}))}{\sum_{\mathbf{V}', \mathbf{h}'} \exp(-E(\mathbf{V}', \mathbf{h}'))},$$

$$E(\mathbf{V}, \mathbf{h}) = -\sum_{i=1}^{m} \sum_{k=1}^{K} \sum_{j=1}^{F} v_i^k W_{ij}^k h_j - \sum_{i=1}^{m} \sum_{k=1}^{K} b_i^k v_i^k - \sum_{j=1}^{F} b_j h_j .$$

The movies with missing ratings do not make any contribution to the energy function.

Learning

$$\Delta W_{ij}^k = \epsilon \frac{\partial \log p(\mathbf{V})}{\partial W_{ij}^k} = \epsilon \left(\langle v_i^k h_j \rangle_{\text{data}} - \langle v_i^k h_j \rangle_{\text{model}} \right).$$

The movies with missing ratings do not make any contribution to the energy function.

The model average is learned using contrastive divergence:

$$\Delta W_{ij}^k = \epsilon \left(\langle v_i^k h_j \rangle_{\text{data}} - \langle v_i^k h_j \rangle_{\tau} \right).$$

τ is the number of steps used in Gibbs sampling.

$\tau = 1$ at the beginning of learning.

τ increases in the process of learning.

Making Predictions (1)

When asked to predict ratings for n movies $q_1, q_2, \ldots, q_n$, we can compute

$$p\left(v_{q_1}^{k_1} = 1, v_{q_2}^{k_2} = 1, \ldots, v_{q_n}^{k_n} = 1 \middle| \mathbf{V}\right)$$

However, this requires us to make K^n evaluations for each user.

Making Predictions (2)

Alternatively, use mean field equations,

$$\hat{p}_j = p\left(h_j = 1 \middle| \mathbf{V}\right) = \frac{1}{1 + \exp\left(-b_j - \sum_{i=1}^{m} \sum_{k=1}^{K} v_i^k W_{ij}^k\right)},$$

$$p\left(v_q^k = 1 \middle| \hat{\mathbf{p}}\right) = \frac{\exp\left(b_q^k + \sum_{j=1}^{F} W_{qj}^k \hat{p}_j\right)}{\sum_{l=1}^{K} \exp\left(b_q^l + \sum_{j=1}^{F} W_{qj}^l \hat{p}_j\right)},$$

$$E\left(v_q\right) = \sum_{k=1}^{K} p\left(v_q^k = 1 \middle| \hat{\mathbf{p}}\right). \quad \text{(slightly less accurate but faster, adopted)}$$

Further Details

- RBM's with Gaussian hidden nodes
- Conditional RBM's: Pay more attention to the movies to be queried, if those movies are known to be queried beforehand.
- Conditional factored RBM's: Factorize the weight matrix to address the size problem of the weight matrix with $M \times K \times F$ (e.g. 9 million for $F = 100, M = 17{,}770, K = 5$ in the Netflix dataset):

$$W_{ij}^k = \sum_{c=1}^{C} A_{ic}^k B_{cj} \qquad (C \ll M \text{ and } C \ll F, \text{e.g. } C = 30).$$

Learning:

$$\Delta A_{ic}^k = \epsilon\left(\left\langle \left[\sum_j B_{cj} h_j\right] v_i^k \right\rangle_{\text{data}} - \left(\left\langle \left[\sum_j B_{cj} h_j\right] v_i^k \right\rangle_{\tau}\right),$$

$$\Delta B_{cj} = \epsilon\left(\left\langle \left[\sum_{ik} A_{ic}^k v_i^k\right] h_j \right\rangle_{\text{data}} - \left(\left\langle \left[\sum_{ik} A_{ic}^k v_i^k\right] h_j \right\rangle_{\tau}\right).$$

Experimental Results

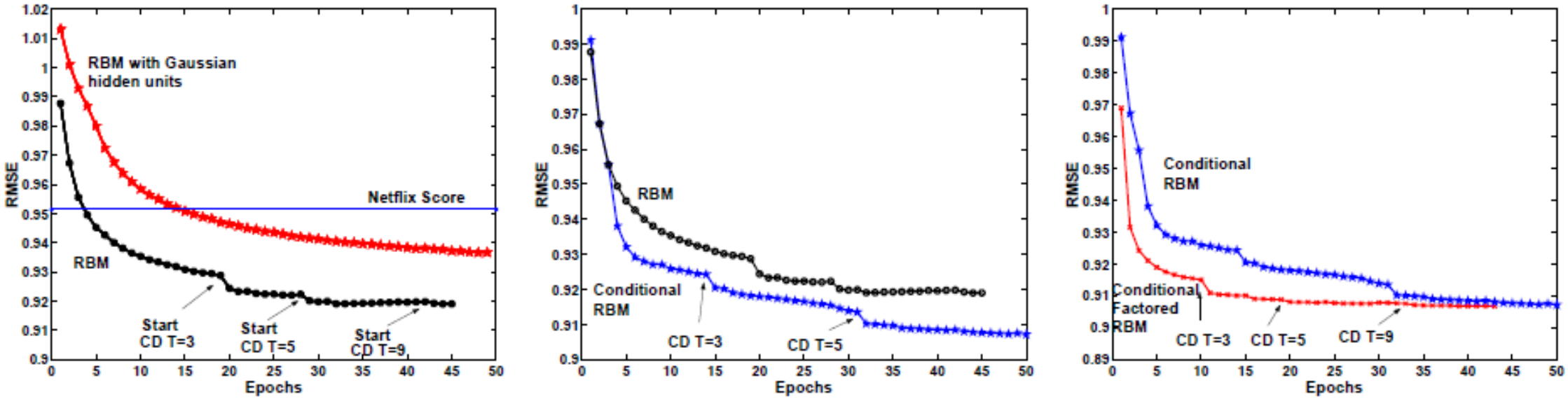


Figure 3. Performance of various models on the validation data. Left panel: RBM vs. RBM with Gaussian hidden units. Middle panel: RBM vs. conditional RBM. Right panel: conditional RBM vs. conditional factored RBM. The y-axis displays RMSE (root mean squared error), and the x-axis shows the number of epochs, or passes through the entire training dataset.